Bài 1.7 – Thống kê mô tả: trung bình, trung vị, phương sai, phân phối, tương quan

> **Giai đoạn 1 – Python cho ML & Data Science** · Thời lượng: 2 buổi · Hình thức mở đầu: *Câu chuyện*

🎯 Sau bài này bạn sẽ

- Chọn đúng giữa trung bình, trung vị, mode để mô tả "giá trị điển hình".
- Hiểu và tự tính được phương sai, độ lệch chuẩn: dữ liệu "ổn định" hay "thất thường".
- Nhận ra các dạng phân phối: chuẩn, lệch phải, lệch trái. Biết dùng log để "nắn" dữ liệu lệch.
- Tự tính được hệ số tương quan, đọc ma trận tương quan, và biết giới hạn của nó.

Chia buổi:

- **Buổi 1:** Mục 1 → 4 (đo trung tâm, đo độ phân tán, phân vị).
- **Buổi 2:** Mục 5 → 9 (phân phối, tương quan, bài tập).

---

1. Câu chuyện: "Lương trung bình công ty tôi là 64,5 triệu"

Một startup đăng tin tuyển dụng: "*Lương trung bình nhân viên: 64,5 triệu/tháng!*"

Bạn nộp đơn, được nhận, và nhận lương 15 triệu. Bạn hỏi đồng nghiệp. Ai cũng 12–22 triệu. Hóa ra bảng lương 10 người là:

```
12  13  14  15  15  16  18  20  22  500
                                    ↑
                                lương CEO
```

```python
import numpy as np
import pandas as pd

luong = pd.Series([12, 13, 14, 15, 15, 16, 18, 20, 22, 500])   # triệu đồng/tháng
print("Trung bình:", luong.mean(), "| Trung vị:", luong.median(), "| Mode:",
luong.mode().tolist())
```

Kết quả mẫu:

```
Trung bình: 64.5 | Trung vị: 15.5 | Mode: [15]
```

Công ty không nói dối. Họ chỉ **chọn con số có lợi cho họ**. 9/10 người nhận dưới 23 triệu, nhưng "trung bình" là 64,5 triệu.

Bài này dạy bạn **không bị lừa bởi con số**, và quan trọng hơn: **chọn đúng con số** khi mô tả dữ liệu cho mô hình ML.

---

2. Đo "trung tâm": giá trị điển hình là bao nhiêu?

2.1. Mean (Trung bình cộng)

- **Định nghĩa:** Tổng mọi giá trị chia cho số lượng.
- **Ví dụ đời thực:** Điểm trung bình học kỳ.

Công thức:

$$\bar{x} = \frac{x_1 + x_2 + ... + x_n}{n}$$

Ký hiệu	Ý nghĩa
$\bar{x}$ (đọc là "x ngang")	Trung bình
$x_1, x_2, ..., x_n$	Từng giá trị
n	Số lượng giá trị

Tính thử: (12 + 13 + 14 + 15 + 15 + 16 + 18 + 20 + 22 + 500) / 10 = 645 / 10 = **64,5**.

2.2. Median (Trung vị)

- **Định nghĩa:** Giá trị **đứng giữa** khi xếp dữ liệu từ nhỏ đến lớn. Một nửa nhỏ hơn, một nửa lớn hơn.
- **Ví dụ đời thực:** "Một nửa số nhân viên lương dưới mức này."

Cách tính:

- Số lượng **lẻ**: lấy phần tử chính giữa.
- Số lượng **chẵn**: trung bình của 2 phần tử giữa.

Tính thử: 10 giá trị (chẵn), 2 phần tử giữa là vị trí 5 và 6: 15 và 16. Trung vị = (15 + 16) / 2 = **15,5**.

2.3. Mode (Yếu vị)

- **Định nghĩa:** Giá trị **xuất hiện nhiều nhất**.
- **Ví dụ đời thực:** Size áo bán chạy nhất. Món được gọi nhiều nhất.
- Dùng được cho cả dữ liệu chữ (bài 1.6 đã dùng để điền ô trống).

2.4. Khi nào dùng cái nào?

Tình huống	Dùng	Vì sao
Dữ liệu cân đối, không outlier (chiều cao, điểm thi)	**Trung bình**	Dùng hết thông tin

Tình huống	Dùng	Vì sao
Dữ liệu lệch hoặc có outlier (lương, giá nhà, thu nhập)	**Trung vị**	Không bị kéo lệch
Dữ liệu chữ / nhóm (màu, size, quận)	**Mode**	Chỉ có nó áp dụng được

Mẹo: Nếu **trung bình lớn hơn nhiều so với trung vị** → có giá trị cực lớn kéo lên → dữ liệu **lệch phải**.

---

3. Đo "độ phân tán": dữ liệu ổn định hay thất thường?

3.1. Vấn đề

Hai quán cà phê có điểm đánh giá 5 ngày gần nhất (thang 10):

```
Quán A: 7  8  8  9  8     → trung bình 8
Quán B: 4 10  6 12  8     → trung bình 8
```

(Quán B có ngày 12 điểm vì được cộng điểm thưởng sự kiện.) Trung bình như nhau. Nhưng bạn sẽ chọn quán nào cho buổi hẹn quan trọng? Quán A **ổn định**, quán B **thất thường**. Ta cần một con số đo sự "thất thường" đó.

3.2. Variance (Phương sai)

- **Định nghĩa:** Trung bình của **bình phương khoảng cách** từ mỗi giá trị đến trung bình.
- **Ví dụ đời thực:** Đo xem các phát bắn rải rác quanh hồng tâm nhiều hay ít.

Công thức (phương sai mẫu):

$$s^2 = \frac{(x_1 - \bar{x})^2 + (x_2 - \bar{x})^2 + \ldots + (x_n - \bar{x})^2}{n - 1}$$

Ký hiệu	Ý nghĩa
s^2	Phương sai
$x_i - \bar{x}$	Khoảng cách từ giá trị thứ i đến trung bình
$(\ldots)^2$	Bình phương: để khoảng cách âm và dương không triệt tiêu nhau, và phạt nặng các điểm xa
$n - 1$	Chia cho n−1 thay vì n (giải thích bên dưới)

Tính thử với quán B (trung bình = 8):

Ngày	x	x − 8	$(x - 8)^2$
1	4	−4	16
2	10	2	4

Ngày	x	x − 8	$(x-8)^2$
3	6	−2	4
4	12	4	16
5	8	0	0
		Tổng	**40**

s^2 = 40 / (5 − 1) = 40 / 4 = **10**.

3.3. Standard Deviation (Độ lệch chuẩn)

- **Định nghĩa:** Căn bậc hai của phương sai. Cho biết "trung bình mỗi giá trị lệch khỏi trung bình khoảng bao nhiêu", **cùng đơn vị** với dữ liệu gốc.
- **Ví dụ đời thực:** "Điểm quán B thường dao động khoảng ±3 điểm quanh mức 8."

Tính thử: s = √10 ≈ **3,16**.

Vì sao cần cả hai? Phương sai có đơn vị là "điểm²", khó hình dung. Độ lệch chuẩn trở về đơn vị "điểm", dễ hiểu.

```
a = np.array([7, 8, 8, 9, 8])
b = np.array([4, 10, 6, 12, 8])
for ten, x in [("Quán A", a), ("Quán B", b)]:
    print(ten, "TB =", x.mean(),
          "| phương sai =", x.var(ddof=1),              # ddof=1: chia cho n-1
          "| ĐLC =", round(x.std(ddof=1), 2))
```

Kết quả mẫu:

```
Quán A TB = 8.0 | phương sai = 0.5 | ĐLC = 0.71
Quán B TB = 8.0 | phương sai = 10.0 | ĐLC = 3.16
```

Quán B thất thường gấp hơn 4 lần (3,16 so với 0,71).

3.4. Cái bẫy `ddof`: NumPy và Pandas tính khác nhau!

```
print(np.var(b), np.var(b, ddof=1), pd.Series(b).var())
```

Kết quả mẫu:

```
8.0 10.0 10.0
```

Thư viện	Mặc định chia cho	Gọi là
NumPy `np.var`, `np.std`	n	Phương sai **tổng thể** (population)
Pandas `.var()`, `.std()`	n − 1	Phương sai **mẫu** (sample)

Vì sao chia n − 1? Khi chỉ có một **mẫu nhỏ** (5 ngày) thay vì **toàn bộ** (mọi ngày từ khi mở quán), mẫu thường "trông ổn định hơn thực tế". Chia cho số nhỏ hơn (n−1) giúp bù lại phần đánh giá thấp đó. Với dữ liệu lớn (hàng nghìn dòng), khác biệt gần như bằng 0. Bạn chỉ cần nhớ: **hai thư viện có mặc định khác nhau**, đừng ngạc nhiên khi số lệch nhau.

3.5. Phân vị (Percentile)

- **Định nghĩa:** Phân vị p% là giá trị mà p% dữ liệu nhỏ hơn nó.
- **Ví dụ đời thực:** "Điểm thi của bạn nằm ở phân vị 90" = bạn giỏi hơn 90% thí sinh.

```
print(luong.quantile([0.25, 0.5, 0.75, 0.9]).tolist())   # phân vị 25%, 50%, 75%, 90%
```

Kết quả mẫu:

```
[14.25, 15.5, 19.5, 69.79999999999983]
```

- Phân vị 50% = 15,5 = **chính là trung vị**.
- 75% nhân viên lương dưới 19,5 triệu. Phân vị 90% bị CEO kéo lên gần 70 triệu.
- Q1 (25%) và Q3 (75%) là nguyên liệu của phương pháp IQR ở bài 1.6.

---

4. Bảng tóm tắt các con số

Con số	Trả lời câu hỏi	Nhạy với outlier?	Pandas
Trung bình	Giá trị điển hình?	**Rất nhạy**	`.mean()`
Trung vị	Giá trị điển hình?	Không	`.median()`
Mode	Giá trị hay gặp nhất?	Không	`.mode()`
Phương sai	Phân tán bao nhiêu? (đơn vị^2)	Rất nhạy	`.var()`
Độ lệch chuẩn	Phân tán bao nhiêu? (cùng đơn vị)	Rất nhạy	`.std()`
IQR	50% ở giữa rộng bao nhiêu?	Không	`.quantile(.75) - .quantile(.25)`

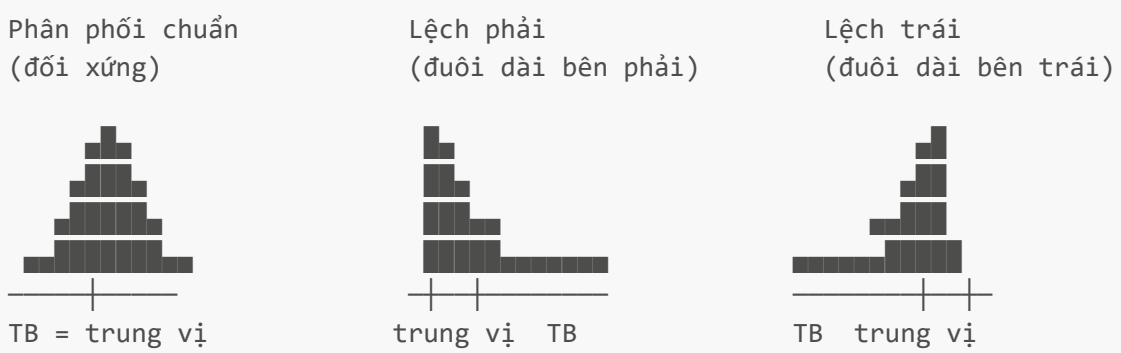
---

5. Phân phối (Distribution)

5.1. Khái niệm

- **Distribution (Phân phối):** Bức tranh cho biết các giá trị nằm ở đâu và nhiều hay ít. Chính là **hình dáng của histogram** (bài 1.5).

5.2. Ba hình dáng hay gặp



5.3. Phân phối chuẩn và quy tắc 68–95–99,7

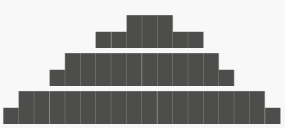
- **Normal Distribution (Phân phối chuẩn):** dạng hình chuông đối xứng, rất phổ biến trong tự nhiên.
- **Ví dụ:** Chiều cao người trưởng thành.

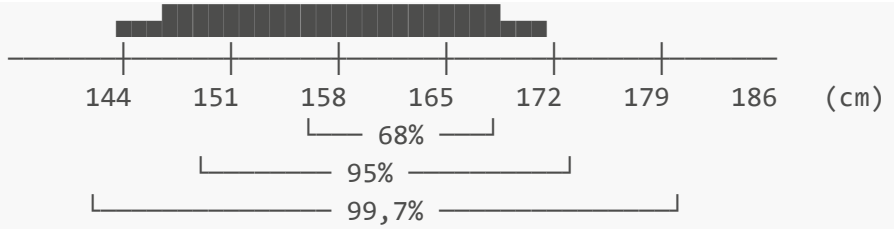
Mô phỏng chiều cao 100.000 người với trung bình 165 cm, độ lệch chuẩn 7 cm:

```
rng = np.random.default_rng(0)
chieu_cao = rng.normal(loc=165, scale=7, size=100_000)   # loc: trung bình, scale: ĐLC
tb, dlc = chieu_cao.mean(), chieu_cao.std()
for k in [1, 2, 3]:
    ti_le = ((chieu_cao > tb - k * dlc) & (chieu_cao < tb + k * dlc)).mean()   # mean của True/False = tỉ lệ
    print(f"Trong khoảng ±{k} ĐLC: {ti_le:.1%}")
```

Kết quả mẫu:

```
Trong khoảng ±1 ĐLC: 68.4%
Trong khoảng ±2 ĐLC: 95.5%
Trong khoảng ±3 ĐLC: 99.7%
```





Nghĩa là: khoảng 95% người cao từ 151 đến 179 cm. Ai cao trên 186 cm (quá 3 ĐLC) là cực hiếm. Đây chính là lý do quy tắc "|z| > 3 là outlier" ở bài 1.6 ra đời. Nhưng nhớ: quy tắc này **chỉ đúng với phân phối chuẩn**.

5.4. Skewness (Độ lệch)

- **Định nghĩa:** Con số đo mức độ lệch của phân phối. ≈ 0 là cân đối; > 0 lệch phải; < 0 lệch trái. Trị tuyệt đối trên 1 thường coi là lệch mạnh.

```
url = "https://download.mlcc.google.com/mledu-
datasets/california_housing_train.csv"
df = pd.read_csv(url)
df.columns = ["kinh_do", "vi_do", "tuoi_nha", "tong_phong", "tong_phong_ngu",
              "dan_so", "so_ho", "thu_nhap", "gia_nha"]

for c in ["gia_nha", "thu_nhap", "dan_so", "tuoi_nha"]:
    print(c, "TB =", round(df[c].mean(), 2),
          "| Trung vị =", round(df[c].median(), 2),
          "| skew =", round(df[c].skew(), 2))        # .skew(): độ lệch
```

Kết quả mẫu:

```
gia_nha TB = 207300.91 | Trung vị = 180400.0 | skew = 0.97
thu_nhap TB = 3.88 | Trung vị = 3.54 | skew = 1.63
dan_so TB = 1429.57 | Trung vị = 1167.0 | skew = 5.19
tuoi_nha TB = 28.59 | Trung vị = 29.0 | skew = 0.06
```

- `tuoi_nha`: trung bình ≈ trung vị, skew ≈ 0 → **cân đối**.
- `dan_so`: skew = 5,19 → **lệch phải rất mạnh**.
- Đúng như mẹo ở mục 2.4: cột nào có trung bình > trung vị thì lệch phải.

5.5. Log transform: "nắn" dữ liệu lệch phải

Vấn đề: Nhiều mô hình ML (như Linear Regression, bài 2.2) học tốt hơn khi dữ liệu không quá lệch. Vài giá trị khổng lồ sẽ "chi phối" mô hình.

Trực giác: Hàm log "nén" số lớn mạnh hơn số nhỏ.

x	log(x + 1)
10	2,4

x	log(x + 1)
100	4,6
1.000	6,9
10.000	9,2

Từ 10 đến 10.000 (gấp 1.000 lần) chỉ còn từ 2,4 đến 9,2. Khoảng cách giữa số lớn bị co lại.

```
import matplotlib.pyplot as plt

df["log_dan_so"] = np.log1p(df["dan_so"])     # log1p(x) = log(x + 1), an toàn khi
x = 0
print("Skew sau log:", round(df["log_dan_so"].skew(), 2))

fig, axes = plt.subplots(1, 2, figsize=(10, 3.5))
axes[0].hist(df["dan_so"], bins=60);     axes[0].set_title("Dân số (gốc)")
axes[1].hist(df["log_dan_so"], bins=60); axes[1].set_title("log(Dân số + 1)")
plt.show()
```

Kết quả mẫu:

```
Skew sau log: -0.97
```

Bạn sẽ thấy: hình trái là một cột cao dồn hết về bên trái, đuôi kéo dài tới 35.000. Hình phải là một **hình chuông** khá đẹp, chỉ còn một đuôi nhỏ bên trái. Độ lệch giảm từ 5,19 xuống còn khoảng −1 (trị tuyệt đối nhỏ hơn 5 lần).

---

6. Tương quan (Correlation)

6.1. Khái niệm

- **Correlation (Tương quan):** Mức độ hai biến số **cùng tăng / cùng giảm theo đường thẳng**.
- **Hệ số tương quan Pearson (r):** con số từ −1 đến +1 đo mức độ đó.
- **Ví dụ đời thực:** Diện tích nhà và giá nhà: nhà càng rộng càng đắt → tương quan dương mạnh.

```
 r ≈ +1              r ≈ 0               r ≈ −1
 (cùng tăng)         (không liên quan     (cái này tăng,
                      theo đường thẳng)    cái kia giảm)
y|       •          y| •   •   •        y|•
 |     •              |   •   •           |  •
 |   •                | •   •    •        |    •
 | •                  |   •   •           |      •
 └──────x             └──────x            └──────x
```

```
Diện tích – giá      Số điện thoại –       Khoảng cách tới
                     chiều cao             trung tâm – giá
```

| |r| | Mức độ (quy ước tham khảo) | |---|---| | 0 – 0,3 | Yếu | | 0,3 – 0,7 | Trung bình | | 0,7 – 1 | Mạnh |

6.2. Công thức

$$r = \frac{\Sigma (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\Sigma(x_i - \bar{x})^2} \times \sqrt{\Sigma(y_i - \bar{y})^2}}$$

Ký hiệu	Ý nghĩa
Σ (đọc "xích-ma")	Tổng của tất cả các số hạng
$(x_i - \bar{x})(y_i - \bar{y})$	Tích độ lệch. **Dương** nếu x và y cùng lớn hơn (hoặc cùng nhỏ hơn) trung bình
Mẫu số	Chia để chuẩn hóa về khoảng [−1, 1], không phụ thuộc đơn vị đo

Trực giác của tử số: Nếu căn nào rộng hơn trung bình cũng đắt hơn trung bình, các tích đều dương → tổng lớn → r gần +1.

6.3. Tính thử bằng tay

5 căn hộ: diện tích (m^2) và giá (tỷ).

Căn	x (m^2)	y (tỷ)	x − 60	y − 3,02	tích	$(x - 60)^2$	$(y - 3{,}02)^2$
1	40	2,0	−20	−1,02	20,4	400	1,0404
2	50	2,6	−10	−0,42	4,2	100	0,1764
3	60	2,9	0	−0,12	0	0	0,0144
4	70	3,6	10	0,58	5,8	100	0,3364
5	80	4,0	20	0,98	19,6	400	0,9604
	TB = 60	TB = 3,02		**Tổng**	**50**	**1000**	**2,528**

- √1000 ≈ 31,623; √2,528 ≈ 1,590.
- r = 50 / (31,623 × 1,590) = 50 / 50,28 ≈ **0,994**. Tương quan dương rất mạnh.

```
dien_tich = np.array([40, 50, 60, 70, 80])
gia = np.array([2.0, 2.6, 2.9, 3.6, 4.0])
dx = dien_tich - dien_tich.mean()          # độ lệch của x
dy = gia - gia.mean()                      # độ lệch của y
r = (dx * dy).sum() / (np.sqrt((dx ** 2).sum()) * np.sqrt((dy ** 2).sum()))
print(round(r, 4), np.corrcoef(dien_tich, gia)[0, 1].round(4))   # tự tính vs
NumPy
```

Kết quả mẫu:

```
0.9944 0.9944
```

6.4. Ma trận tương quan trên dữ liệu thật

```
print(df[["thu_nhap", "tuoi_nha", "dan_so", "gia_nha"]].corr().round(2))
```

Kết quả mẫu:

```
           thu_nhap  tuoi_nha  dan_so  gia_nha
thu_nhap       1.00     -0.12   -0.00     0.69
tuoi_nha      -0.12      1.00   -0.30     0.11
dan_so        -0.00     -0.30    1.00    -0.03
gia_nha        0.69      0.11   -0.03     1.00
```

Đọc:

- Đường chéo luôn = 1 (một biến tương quan hoàn hảo với chính nó).
- Bảng **đối xứng**: tương quan(A, B) = tương quan(B, A).
- `thu_nhap` – `gia_nha` = 0,69: trung bình, gần mạnh. Đặc trưng quý nhất.
- `tuoi_nha` – `dan_so` = −0,30: khu nhà mới thường đông dân hơn (khu đô thị mới xây nhiều căn hộ?).

Trong ML, câu hỏi đầu tiên luôn là: **đặc trưng nào tương quan mạnh nhất với cột đích?**

```
print(df.drop(columns="log_dan_so").corr()
["gia_nha"].sort_values(ascending=False).round(3))
```

Kết quả mẫu:

```
gia_nha           1.000
thu_nhap          0.692
tong_phong        0.131
```

```
tuoi_nha          0.107
so_ho             0.061
tong_phong_ngu    0.046
dan_so           -0.028
kinh_do          -0.045
vi_do            -0.145
Name: gia_nha, dtype: float64
```

6.5. Ba cái bẫy của tương quan ⚠️

Bẫy 1 – Tương quan không phải nhân quả. (Đã gặp ở bài 1.5: nhà cũ đắt hơn vì vị trí, không phải vì cũ.) Ví dụ kinh điển: số kem bán ra tương quan với số người chết đuối. Kem không gây chết đuối. Cả hai cùng tăng vào mùa hè.

Bẫy 2 – r chỉ đo quan hệ ĐƯỜNG THẲNG.

```
x = np.linspace(-3, 3, 61)    # 61 điểm cách đều từ -3 đến 3
y = x ** 2                    # y phụ thuộc HOÀN TOÀN vào x (hình parabol)
print(round(np.corrcoef(x, y)[0, 1], 4))
```

Kết quả mẫu:

```
0.0
```

y được tính 100% từ x, nhưng r = 0! Vì quan hệ là đường cong chữ U, không phải đường thẳng. Nhớ lại bộ B của Anscombe (bài 1.5). **r ≈ 0 không có nghĩa là "không liên quan".** Luôn vẽ scatter để kiểm tra.

Tương tự, `kinh_do` và `vi_do` có r gần 0 với giá nhà, nhưng bản đồ ở bài 1.5 cho thấy **vị trí cực kỳ quan trọng**. Quan hệ của vị trí với giá phức tạp, không phải đường thẳng. Các mô hình như Random Forest (bài 2.10) sẽ khai thác được những quan hệ này.

Bẫy 3 – Outlier làm méo r. Một điểm cực đoan có thể tạo ra hoặc phá hủy tương quan. Khi nghi ngờ, dùng **tương quan Spearman**: tính tương quan trên **thứ hạng** thay vì giá trị gốc, nên ít bị outlier ảnh hưởng và bắt được cả quan hệ "cùng tăng" không thẳng.

```
print(df[["thu_nhap", "gia_nha"]].corr(method="spearman").round(3).iloc[0, 1])
```

Kết quả mẫu:

```
0.681
```

Gần với Pearson (0,692) → quan hệ thu nhập – giá nhà khá "thẳng" và ổn định.

---

7. Ứng dụng trong ML

Bạn tính	Để quyết định
Trung bình vs trung vị	Dữ liệu có lệch không? Điền ô trống bằng gì?
Độ lệch chuẩn	Có cần chuẩn hóa không? (bài 1.11)
Skewness	Có cần log transform không?
Phân vị, IQR	Đâu là outlier?
Tương quan với cột đích	Đặc trưng nào hứa hẹn nhất?
Tương quan giữa các đặc trưng	Có hai cột "nói cùng một điều" không?

---

8. Bài tập

1. Điểm 8 bài kiểm tra của Minh: `[6.5, 7.0, 7.5, 8.0, 8.0, 8.5, 9.0, 2.0]` (bài 2.0 do Minh bị ốm). Tính trung bình, trung vị, mode, độ lệch chuẩn. Con số nào mô tả đúng nhất năng lực của Minh? Tính lại trung bình và ĐLC khi bỏ bài 2.0.
2. **Tính tay** phương sai và độ lệch chuẩn của quán A `[7, 8, 8, 9, 8]`. Đối chiếu với kết quả ở mục 3.3.
3. Với dataset California: so sánh **hệ số biến thiên** (CV = ĐLC / trung bình) của `gia_nha` và `thu_nhap`. Cột nào "thất thường" hơn **tương đối**? (CV giúp so sánh độ phân tán của hai cột khác đơn vị.)
4. Tạo cột `phong_moi_ho = tong_phong / so_ho`. Tương quan Pearson của nó với `gia_nha` lớn hơn hay nhỏ hơn của `tong_phong`? Tính thêm Spearman. Giải thích sự khác biệt.

Đáp án

> Mẹo Notion: chọn tiêu đề "Đáp án" → `Turn into → Toggle heading 3`.

```
# 1
diem = pd.Series([6.5, 7.0, 7.5, 8.0, 8.0, 8.5, 9.0, 2.0])
print(diem.mean(), diem.median(), diem.mode().tolist(), round(diem.std(), 2))
# 7.0625 7.75 [8.0] 2.19
print(diem.drop(7).mean().round(3), round(diem.drop(7).std(), 2))
# 7.786 0.86
# Trung vị 7,75 mô tả đúng năng lực hơn. Một bài 2.0 kéo trung bình xuống 7,06
# và làm ĐLC phình từ 0,86 lên 2,19.
```

2. TB = 8. Độ lệch: −1, 0, 0, 1, 0. Bình phương: 1, 0, 0, 1, 0. Tổng = 2. Phương sai = 2 / (5 − 1) = **0,5**. ĐLC = √0,5 ≈ **0,71**. Khớp!

```
# 3
print(round(df["gia_nha"].std() / df["gia_nha"].mean(), 3),
      round(df["thu_nhap"].std() / df["thu_nhap"].mean(), 3))
# 0.559 0.491 → giá nhà phân tán tương đối nhiều hơn thu nhập một chút.

# 4
```

```
df["phong_moi_ho"] = df["tong_phong"] / df["so_ho"]
print(round(df["phong_moi_ho"].corr(df["gia_nha"]), 3),          # 0.15
      round(df["tong_phong"].corr(df["gia_nha"]), 3))            # 0.131
print(round(df["phong_moi_ho"].corr(df["gia_nha"], method="spearman"), 3))   #
0.265
# Đặc trưng tự tạo tương quan tốt hơn cột gốc một chút.
# Spearman (0.265) cao hơn Pearson (0.15) đáng kể → có outlier cực lớn của
phong_moi_ho
# (đã thấy ở bài 1.5) đang làm Pearson bị "méo". Xử lý outlier sẽ giúp ích.
```

---

9. Flashcard

Mặt trước	**Mặt sau**
Trung bình vs trung vị, cái nào chịu outlier tốt?	Trung vị.
Trung bình > trung vị nhiều gợi ý gì?	Dữ liệu lệch phải (có giá trị cực lớn).
Phương sai là gì?	Trung bình bình phương khoảng cách đến trung bình.
Vì sao cần độ lệch chuẩn khi đã có phương sai?	ĐLC cùng đơn vị với dữ liệu, dễ hiểu.
`np.std` vs `pd.Series.std` khác gì?	NumPy chia n, Pandas chia n−1 (mặc định).
Quy tắc 68–95–99,7?	Với phân phối chuẩn: 68%, 95%, 99,7% dữ liệu nằm trong ±1, ±2, ±3 ĐLC.
Khi nào dùng log transform?	Dữ liệu lệch phải mạnh (giá, dân số, thu nhập).
r = 0 nghĩa là không liên quan?	Không! Chỉ là không liên quan theo đường thẳng.
Khi nào dùng Spearman?	Có outlier, hoặc quan hệ "cùng tăng" nhưng không thẳng.

Bài tiếp theo: 1.8 – Toán: vector và ma trận.